



Copilot Workspace etc.

Don Syme, GitHub Next





GitHub Next

Investigating the future of software development

Research, Build, Balance, Ship, Learn



The Situation

"Any task, any software"

"Make github.com AI-rich, today"

"Make github.com a place to get coding done"

"Bring natural language to the IDE"

"What does it mean to program with words?"



The Situation

Software activity begins with words and intent

Before a code change, before a pull request, there are words - a discussion, a bug report, a feature request, an engineering task, a meeting, a design document, an idea.



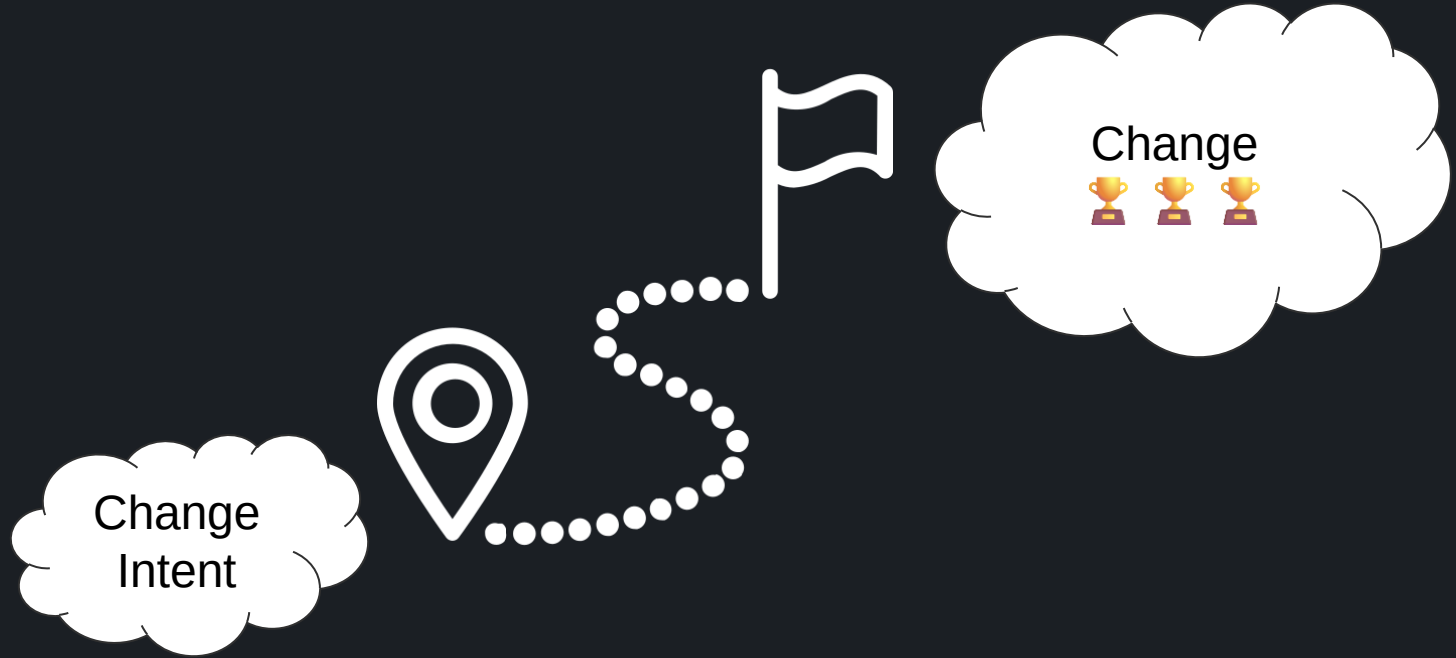
A Journey, Together



A Journey, Together



A Journey, Together



A Journey, Together



A Journey, Together

The original GitHub Copilot offers completion of code in your editor

- You are already located at a pinpoint of change
- You accept/reject small bits
- You move your cursor around
- You have to type quite a lot
- Copilot sort of infers what you are trying to do

This is magical: a finely honed surgical assistant



A Journey, Together

Changing perspective

Pinpoint Surgery
Operation



Whole

Single Point of Change
Repository



Whole



Implicit Task



A Journey, Together

"Add a refresh button to the main page"

"Create end-to-end tests using Playwright"

"Set up continuous integration using GitHub Actions"

"Set up production resources in Azure using Terraform"

"Read coverage report and add unit tests to repo"

"Create a website for UK house prices"



The Need

Contextual. Aware of the context — repository, issue, pull request

Assistive. Navigate unfamiliar tasks, augmenting your development skills

Pervasive. Ready and waiting for you, wherever software change begins, on every issue in every repository

Iterative. Empowers you to check, review, refine AI-generated outputs - you are in control

Collaborative. You share sessions with your team, publish links to your sessions.

Configurable. Integrate into workflows, e.g. via deep links

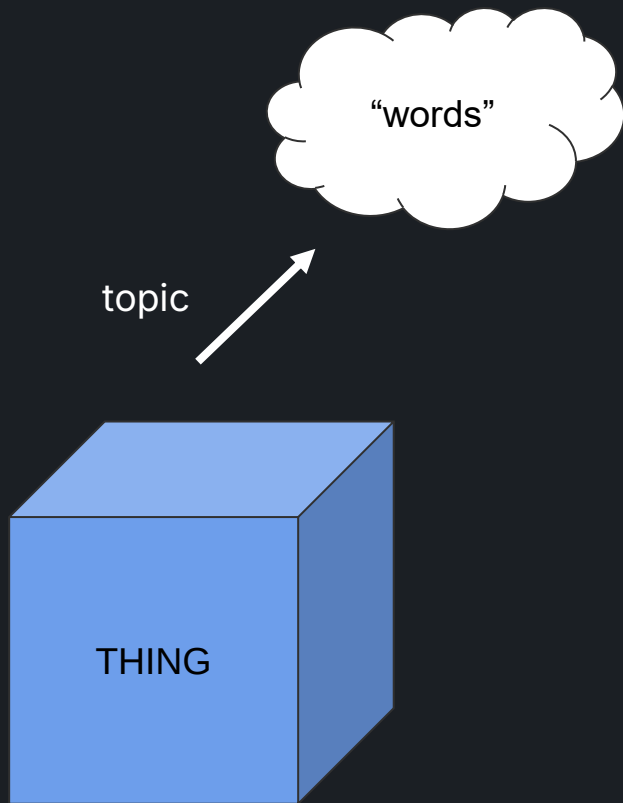


A note on evaluation methodology

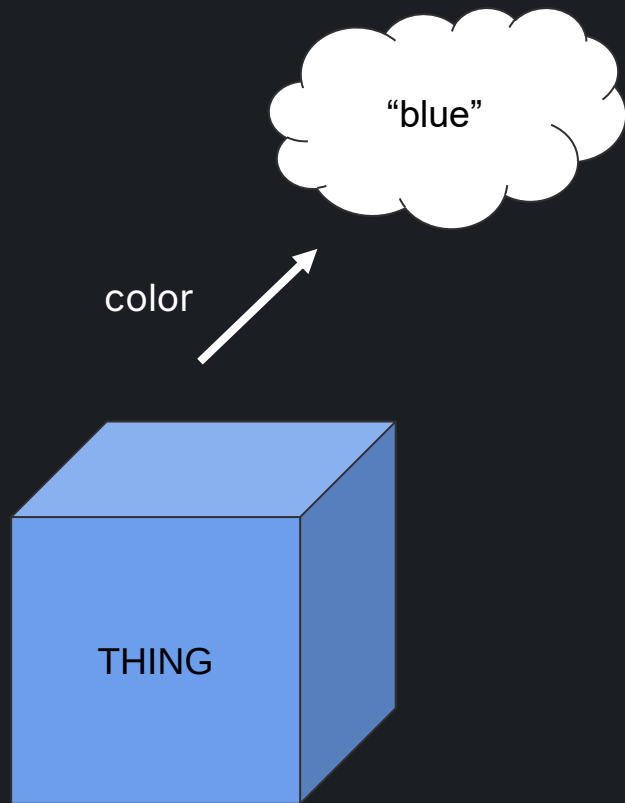
- We don't need to write papers. Our measure is ultimately influence on delivered supported product design
- Primary techniques
 - Simulated walkthroughs of development activities
 - Dogfooding
 - “Baseline” test suite over ~50 tasks, capturing current state and eyeball impact of changes
 - Some benchmarking
 - Some qualitative user testing
 - Release to early adoption waitlist and collect feedback



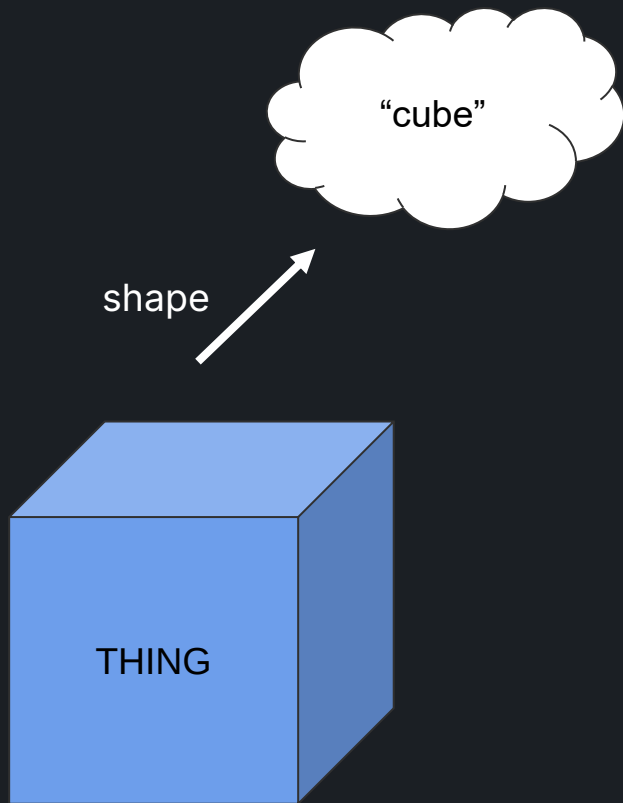
The Original Concept - Extract, Edit, Apply



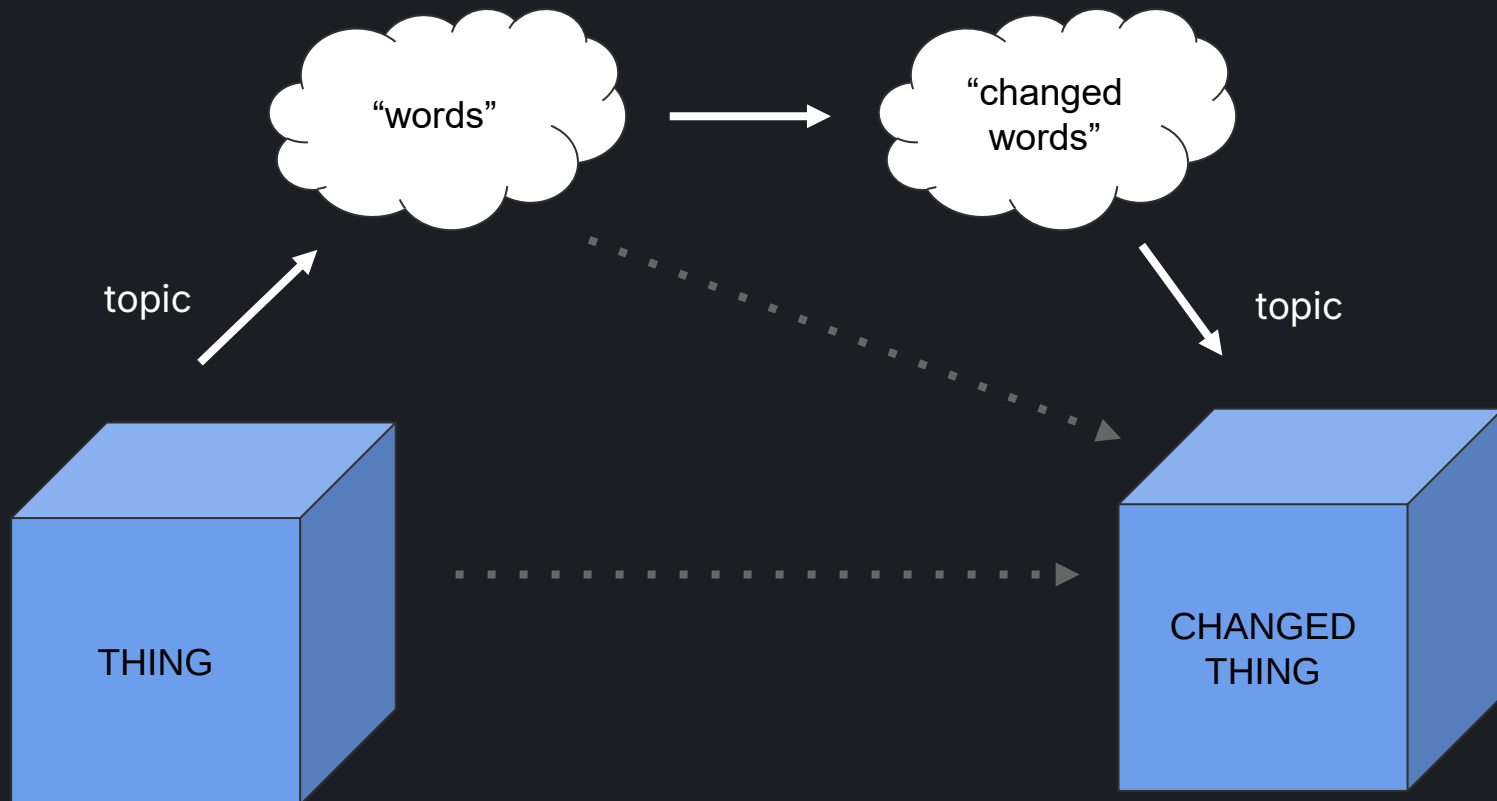
The Original Concept - Extract, Edit, Apply



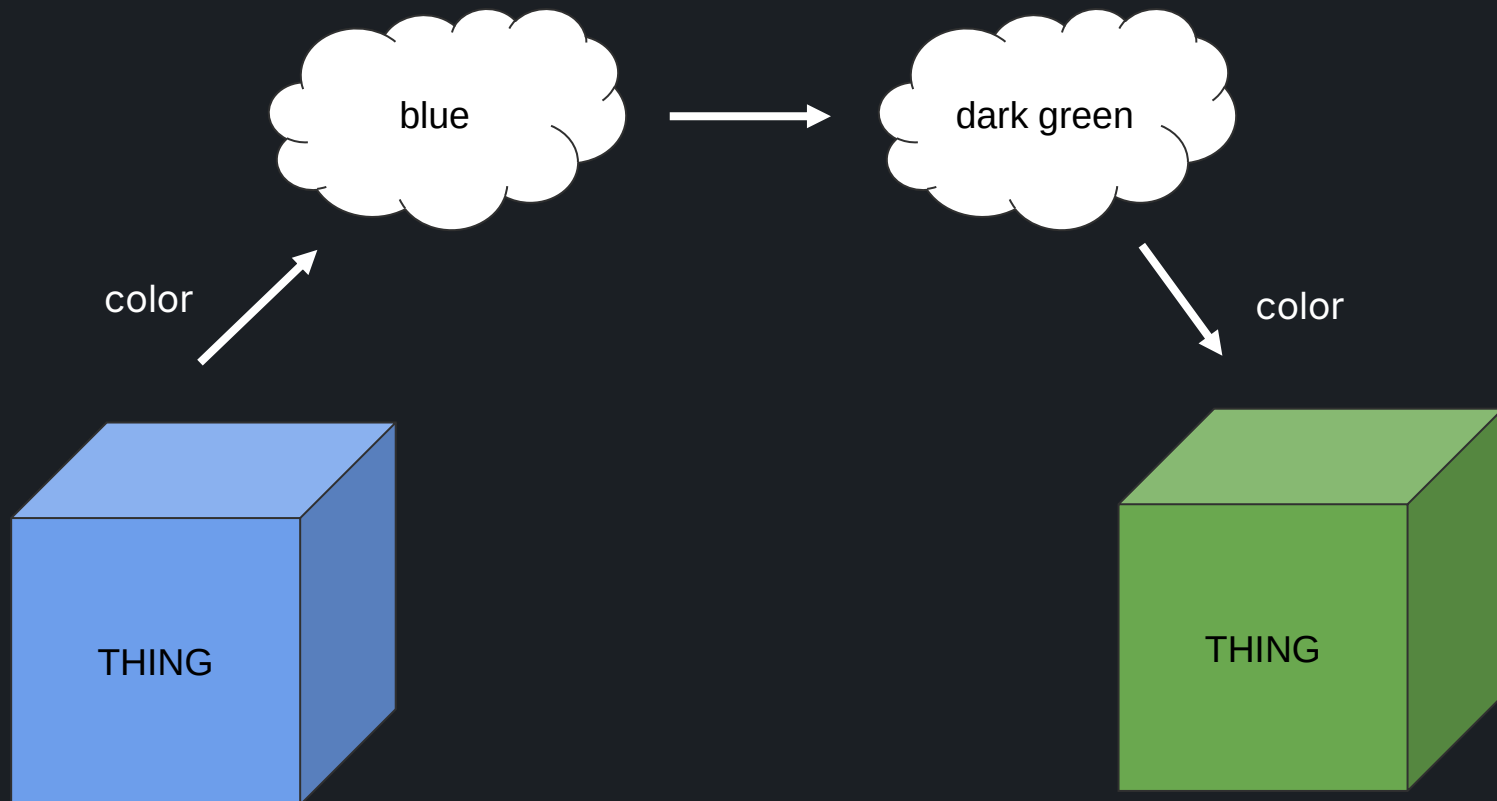
The Original Concept - Extract, Edit, Apply



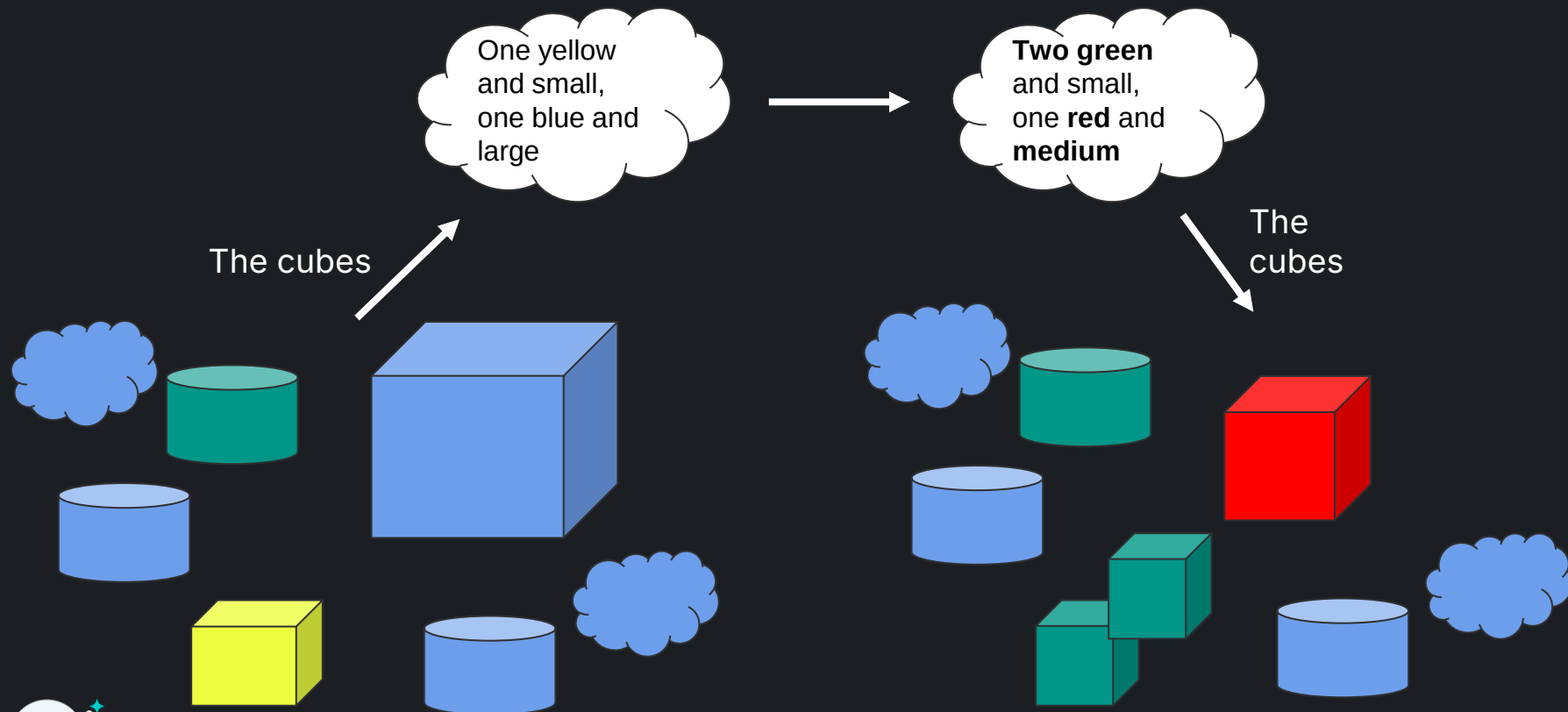
The Original Concept - Extract, Edit, Apply



The Original Concept - Extract, Edit, Apply



The Original Concept - Extract, Edit, Apply



Principles

- Change is described in **natural language**
- "Specifications" are **ephemeral extracts**
- Embraces the **fluidity** and **power** of **words**
- The model both **hides** and **fills in** the details
- There are **many possible extracts**
 - Word clouds
 - Written specs
 - Formal notations
 - Property lists
 - Programming by example
 - Tests



Demo - Issue Solving



Demo - Repo Tasks



Demo - Repo Creation



The Dance of Alignment

The human and AI-tooling are two highly stateful machines/animals.

- For the human, state = psychological matrix of perception, understanding...
- For the AI, state = the available context state

Each state “evolves” in the course of the journey

- Each must “understand” the moves the other makes
- Neither must get too far ahead of the other
- Each move/action can be seen as an “offering” to the other, a *pas de deux*

Mis-alignment can be observed qualitatively (“WTF?”), sometimes measured.



From Agents to Co-agents

AI tooling is an endless sequence of divergence and re-convergence between the human and the AI. This is the "**Co**" in "**Copilot**".

An AI **agent** is usually conceptualised as autonomous. We must discard this thinking, and instead embrace a variation of the concept suited for integration into **Copilot**: **co-agents**.

A **co-agent** is a **controllable**, **guidable**, **cooperative**, **self-re-aligning** unfolding of the reasoning and decision points of an agent.

If an autonomous agent has progress $A \rightarrow B \rightarrow C$, the co-agent is one that can

- propose step A to the human - or multiple possibilities A1, A2 etc.
- receive confirmation of A **and/or adjustment to take step A'**
- etc..



From Agents to Co-agents

- Co-agents don't offer more in a step than a human can digest
- Each significant offering from a co-agent is controllable
- Those controls apply to all downstream offerings
- Co-agents attempt to maintain the integrity of their offerings under further human supervision and control



Six Dimensions of Alignment

Achieving repeated “Alignment” between the AI and Human is crucial.

Six dimensions affecting alignment are

- the **size** of each “offering” from the AI
- the **quality** of the offering
- the **time** to offering
- the **UX** used to show it
- the ease of **accept/ignore/control**
- the **human capacity** to check and correct the offering

Every dimension matters. Not all are under the control of the tool or model developer.



AI Offerings

- Question answering/Idea generation
- Question asking
- Spec writing + revision
- Planning + revision
- Implementation + revision
- Code completions
- Runtime Commands ("build the VSCode extension")
- PR summaries



Control Techniques

- Clarification edits to task
- Edit AI offerings
- Choose from ideas
- Reject/Accept/Re-roll
- Revision Commands ("also add tests")
- Runtime Commands ("build the VSCode extension")



Content selection

- Crucial for almost every AI offering
- Need to reduce 100,000+ possible files to ~20 files
 - Phase 1: AI-generated regexp codesearch
 - Phase 2: algorithmic ranking, to ~5000 files
 - Phase 3: Divide-and-conquer LLM ranking based on filenames and tiny bits of content
- File-based, not snippet-based
- Refined at later stages, e.g. per-file once plan is known
- Evaluated by SWE and other benchmarks



Achieving Iteration is hard!

- Early CW had four modal stages
 - **Task → Spec → Plan → Code**
- Now entirely iterative
 - Task+Ideas → Task+Ideas
 - Task+Code → Task+Code
- A major challenge in all AI tooling
 - Less is more
 - Needs very careful UX design



Fast file rewriting

- We use "speculative decoding" = "predicted outputs"
- Uses logprob analysis to do model-pipelined lookahead in text to determine first edit point



The Future of Programming

- Software is **created** using words
- Software is **changed** using words
- As of 2025, **Code** is still the permanent notation of record, but **Specs** can be **ephemeral**
 - The programming language used is still critical (like the format used for an image or document is critical)



Give it a go!

- Available to all Copilot subscribers
- Now including enterprise accounts
- <https://githubnext.com/projects/copilot-workspace>





GitHub Next

Investigating the future of
software development

githubnext.com

